

Operating on Data in Pandas

Python Data Science Handbook — Chapter 3, Section 3.3

Ufuncs that keep your labels, and the index alignment that turns “mismatched data” from a bug into a feature.

1 Why this section matters

Background & Context

By now you know that a `Series` is a 1-D array with labels and a `DataFrame` is a table of label-aligned columns. This section answers the obvious next question: *what happens when you do math on them?*

The short version is that Pandas inherits NumPy’s two superpowers and then adds a twist:

- **From NumPy:** *vectorization via universal functions* (ufuncs). You write `np.exp(x)` once and a fast compiled loop runs over every element. No Python `for` loop, no per-element overhead.
- **Pandas’ twist:** those operations *preserve and align labels*. A ufunc applied to a `Series` hands you back a `Series` with the same index. A binary operation between two objects first *lines up their labels* and only then does the arithmetic.

That second point is the whole story of this section. It is also where missing values (§3.4) sneak in — so we will meet NaN as a natural by-product, not an accident.

A two-minute refresher on ufuncs and vectorization

Python Refresher

A *ufunc* (“universal function”) is a NumPy function that operates *elementwise* on arrays. The reason it matters is speed. Compare summing reciprocals two ways:

- **Pure Python loop:** the interpreter dispatches on the type of every element, boxes/un-boxes each number, and walks a Python-level loop. For a million elements this is painfully slow.
- **Vectorized ufunc:** `1.0 / arr` pushes the entire loop down into a single precompiled C routine that runs over a contiguous block of memory. Often 50–100× faster.

The mental rule: *if you find yourself writing a Python `for` loop over array elements, there is probably a ufunc that does it faster.*

The convention, as always:

```
import numpy as np
import pandas as pd
```

2 Unary ufuncs: labels ride along

A *unary* ufunc takes one array and returns one array of the same shape: `np.exp`, `np.sin`, `np.log`, `np.sqrt`, and so on. Apply one to a `Series` or `DataFrame` and you get the same kind of object back, with its index and columns untouched.

Let us build a small `Series` of random integers to play with:

```
rng = np.random.default_rng(7)
ser = pd.Series(rng.integers(0, 10, 4),
                 index=['w', 'x', 'y', 'z'])
print(ser)
```

```
w      5
x      9
y      0
z      4
dtype: int64
```

Now apply a ufunc. NumPy is perfectly happy to receive a Pandas object:

```
print(np.exp(ser))
```

```
w      148.413159
x      8103.083928
y         1.000000
z       54.598150
dtype: float64
```

Key Idea

The index `['w', 'x', 'y', 'z']` survived the trip. NumPy computed `np.exp` on the underlying values array (fast, in C), and Pandas re-wrapped the result with the *same* labels. You never have to manually carry the index along — it is glued to the data.

The same holds for a `DataFrame`: the operation hits every cell and both axes' labels are preserved.

```
df = pd.DataFrame(rng.integers(0, 10, (3, 2)),
                  columns=['A', 'B'],
                  index=['r0', 'r1', 'r2'])
print(np.sin(df * np.pi / 4))
```

	A	B
r0	0.707107	1.000000
r1	-0.707107	0.000000
r2	1.000000	-0.707107

Intuition

Think of a Pandas object as a *box* with two parts: a NumPy values array inside, and a label sticker (index, columns) on the outside. A unary ufunc reaches inside, transforms the numbers, and puts the same sticker back on. Nothing about *which* row or column a number belongs to ever changes — only the numbers do.

3 Binary operations and index alignment in Series

Unary ufuncs are easy because there is only one set of labels. The interesting case is a *binary* operation — $+$, $-$, $*$, $/$ — between *two* labeled objects. What if their indices do not match?

This is where Pandas departs sharply from NumPy. In NumPy, $a + b$ adds element 0 to element 0 by *position*; if the shapes differ it errors. In Pandas, addition is done by *label*: Pandas first forms the **union** of the two indices, lines up matching labels, and only then adds.

A worked example. Suppose we track land area for one set of regions and population for an overlapping but not identical set:

```
area = pd.Series({'Alpha': 4000, 'Bravo': 9000,
                  'Charlie': 2500},
                 name='area')
pop  = pd.Series({'Bravo': 8.1e5, 'Charlie': 2.3e5,
                  'Delta': 5.0e5},
                 name='pop')
print(pop / area)
```

```
Alpha      NaN
Bravo      90.000000
Charlie     92.000000
Delta      NaN
dtype: float64
```

Read that result carefully. The index of the output is {Alpha, Bravo, Charlie, Delta} — the *union* of the two inputs. Where a label appears in only one of the two operands (Alpha has area but no pop; Delta has pop but no area), the result is NaN, “not a number,” Pandas’ marker for missing data.

Key Idea

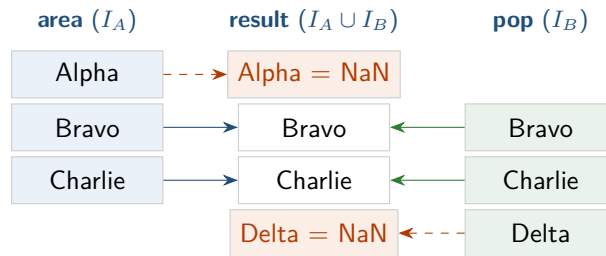
The alignment rule, stated once. For a binary op between Series A and B , the result index is the set union of their labels,

$$I_{\text{result}} = I_A \cup I_B,$$

the arithmetic is applied label-by-label on the matched entries, and any label present in only one operand yields NaN. Order does not matter: $A + B$ and $B + A$ share the same result index.

3.1 Picture: aligning two indices

The diagram below shows the union at work. Solid arrows pair up labels that exist on both sides; the dashed entries are present on only one side, so they fall through to NaN.



Common Pitfall

A single stray NaN *propagates*. Downstream sums, means, and especially comparisons can silently inherit it (and NaN is never equal to anything, even itself: `np.nan == np.nan` is **False**). If an arithmetic result has unexpected NaN holes, suspect a label mismatch before you suspect your math.

3.2 Filling instead of dropping: the object-method form

NaN is the right default — “I genuinely have no value here” — but sometimes a missing operand should be treated as a known quantity (often 0). For that, every operator has a *method* twin that accepts a `fill_value`:

Operator	Method form
<code>a + b</code>	<code>a.add(b)</code>
<code>a - b</code>	<code>a.sub(b)</code>
<code>a * b</code>	<code>a.mul(b)</code>
<code>a / b</code>	<code>a.truediv(b)</code>

The method form takes a keyword argument:

```
print(area.add(pop, fill_value=0))
```

```
Alpha      4000.0
Bravo     819000.0
Charlie    232500.0
Delta     500000.0
dtype: float64
```

Intuition

`fill_value` does *not* mean “replace NaN with this after the fact.” It means: *before* the operation, pretend any label missing from one operand has this value there. So Alpha (missing from `pop`) is treated as $4000 + 0$, and Delta (missing from `area`) as $0 + 500000$. The union still drives the index; the fill value just plugs the holes *before* the arithmetic so no NaN survives.

4 Index alignment in DataFrames

Everything above generalizes cleanly to two dimensions, except now there are *two* axes to align — rows *and* columns — and Pandas aligns both at once.

```
rng = np.random.default_rng(0)
left = pd.DataFrame(rng.integers(0, 10, (2, 2)),
                    columns=['A', 'B'])
right = pd.DataFrame(rng.integers(0, 10, (3, 3)),
                    columns=['B', 'A', 'C'])
print(left + right)
```

	A	B	C
0	6.0	13.0	NaN
1	11.0	7.0	NaN
2	NaN	NaN	NaN

Two things happened simultaneously. The *columns* were aligned by name: A and B appear in both, so they add; C exists only in *right*, so its whole column is NaN. The *rows* were aligned by index: rows 0 and 1 exist in both, but row 2 only exists in *right*, so that whole row is NaN. Note also that *right*'s columns were given in the order B, A, C — alignment is by *label*, so the swapped order is irrelevant and the result comes out in sorted union order.

Key Idea

For a DataFrame binary op, the result's row index is $I_{\text{left}}^{\text{rows}} \cup I_{\text{right}}^{\text{rows}}$ and its columns are $I_{\text{left}}^{\text{cols}} \cup I_{\text{right}}^{\text{cols}}$. Both unions are taken independently, so a cell is non-NaN only if *both* its row label and its column label are present in *both* frames. The same `fill_value` trick (`left.add(right, fill_value=0)`) works here too.

5 Operations between a DataFrame and a Series: broadcasting

The last case mixes dimensions: a 2-D DataFrame with a 1-D Series. To predict the result, recall how NumPy *broadcasts* a 2-D array against a 1-D one.

Python Refresher

NumPy broadcasting, in one line. When you subtract a 1-D array from a 2-D array, NumPy “stretches” the 1-D array across the matching axis. By default a length- n row vector is matched against the *columns* of an $m \times n$ array and subtracted from *each row*:

$$M_{ij} - v_j \quad \text{for every row } i.$$

That is a *row-wise* operation: the vector lines up with the columns and sweeps down the rows.

Pandas follows the same default. Subtracting a Series from a DataFrame matches the Series' index against the DataFrame's *columns* and applies the op row by row.

```
grid = pd.DataFrame(rng.integers(1, 10, (3, 3)),
                    columns=['Q', 'R', 'S'])
print(grid - grid.iloc[0])      # subtract the first row from all rows
```

	Q	R	S
0	0	0	0
1	-3	2	-1
2	4	-1	5

Here `grid.iloc[0]` is a `Series` indexed by the column names `['Q', 'R', 'S']`; it lines up with the columns and is subtracted from every row (so row 0 minus itself is all zeros). This is the default and it is what you usually want for “center each column” style operations.

5.1 Going column-wise with `axis=0`

What if you instead want to operate *down* a column — subtract a per-row quantity from every column? Then you must say so explicitly, using the method form and `axis=0`:

```
col = grid['R']                # a Series indexed by ROW labels
print(grid.subtract(col, axis=0))  # subtract col from every column
```

	Q	R	S
0	...	0	...
1	...	0	...
2	...	0	...

The R column comes out all zeros because we subtracted R from itself, while every other column has R subtracted row by row. The argument `axis=0` reads as “align this `Series` along axis 0 (the rows / index) instead of the columns.”

Intuition

The mnemonic: a `Series` can only line up with *one* of the two axes, and you are telling Pandas which.

- **Default (no axis):** the `Series` index $\rightarrow$ `DataFrame` *columns*. Broadcast *down the rows*. Use when the `Series` is “a row.”
- **`axis=0`:** the `Series` index $\rightarrow$ `DataFrame` *rows*. Broadcast *across the columns*. Use when the `Series` is “a column.”

And crucially: in *both* directions, labels are still matched, not positions. A `Series` entry whose label is missing from the chosen axis produces `NaN`, exactly as before.

6 The tie-in to missing data

Notice the recurring character in every example above: `NaN`. It showed up the moment two label sets failed to match perfectly — in `Series` arithmetic, in `DataFrame` arithmetic, and in broadcasting.

Key Idea

This is by design, and it motivates the next section. Because Pandas aligns on *labels*, “this label exists on one side but not the other” is a completely ordinary situation — the kind of thing that happens constantly when you join two real datasets. Rather than crash, Pandas fills the gap with NaN and keeps going. That makes missing data a *first-class, expected* citizen of the type system, which is precisely why §3.4 (“Handling Missing Data”) is devoted to detecting, filling, and dropping these holes.

Section recap

- **Unary ufuncs** (`np.exp`, `np.sin`, ...) act elementwise and *preserve* the index/columns: same-shape object out, labels intact.
- **Binary ops on Series** align on the *union* of indices, $I_{\text{result}} = I_A \cup I_B$; labels present on only one side become NaN. Order is irrelevant.
- **Method forms** (`add`, `sub`, `mul`, `truediv`) accept `fill_value` to plug holes *before* the operation, so no NaN survives.
- **DataFrame ops** align rows *and* columns simultaneously, each by its own union; a cell is non-NaN only if both labels exist in both frames.
- **DataFrame $\pm$ Series** broadcasts *row-wise* by default (Series index $\rightarrow$ columns); pass `axis=0` to go *column-wise* (Series index $\rightarrow$ rows). Labels are matched in either direction.
- Every mismatch yields NaN — not a bug but the on-ramp to missing-data handling in §3.4.